



Tema 2 Formularios



Tipo de campo

El primer paso es definir el **tipo de campo** para cada elemento del formulario. Esto se realiza estableciendo un valor para el atributo `inputType`.

Aunque no realiza validaciones, determina el tipo de teclado que se muestra, como el teclado numérico para un campo numérico.

EditText

En Android, EditText es un componente de interfaz de usuario que permite al usuario ingresar y editar texto. Los EditText especiales de Material Design proporcionan funcionalidades adicionales y una apariencia más moderna.

Veremos primero los `editText`

EditText Básico

El código crea un EditText básico con una sugerencia (hint) que aparece cuando el campo está vacío.

```
<EditText
    android:id="@+id/editText"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Ingresa su texto" />
```

TextInputEditText (Material Design)

El código anterior utiliza **TextInputLayout** junto con **TextInputEditText** para crear un EditText de Material Design. Esto permite mostrar una etiqueta flotante que se desplaza cuando el usuario comienza a escribir.

```
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/textInputLayout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/textInputEditText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Ingrese su texto" />
</com.google.android.material.textfield.TextInputLayout>
```

Atributos Importantes de TextInputLayout

app:passwordToggleEnabled="true": Permite la revelación de contraseñas.

app:counterEnabled="true": Muestra un contador de caracteres.

app:counterMaxLength="10": Establece la longitud máxima permitida.

app:fontFamily="@font/my_custom_font"

app:textSize="16sp"

app:helperText="Ingrese su nombre completo"

```
<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_marginBottom="10dp"
    android:id="@+id/layout_edit_text_tamano_finca"
    android:hint="Tamaño de la finca"
    app:helperText="Obligatorio"
    style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
>

<com.google.android.material.textfield.TextInputEditText
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="number"
    android:fontFamily="@font/futura"
    android:textSize="20dp"
    android:id="@+id/tamano_finca"
/>

</com.google.android.material.textfield.TextInputLayout>
```

Tamaño de la finca

Obligatorio

InputType

Tipos comunes:

Texto (**text**): `TextInputEditText` es la variante básica para entrada de texto.

Contraseña (**password**): `TextInputEditText` con `inputType="textPassword"` para ocultar los caracteres.

Correo electrónico (**email**): `TextInputEditText` con `inputType="textEmailAddress"` para facilitar la entrada de direcciones de correo electrónico.

Número (**number**): `TextInputEditText` con `inputType="number"` para entradas numéricas.

Fecha (**date**): `TextInputEditText` con `inputType="date"` para entradas de fecha.

Variantes en Material:

Texto con esquinas redondeadas: `Widget.MaterialComponents.TextInputLayout.FilledBox.ExposedDropDownMenu` para un aspecto de caja rellena con esquinas redondeadas.

Contraseña visible (`passwordToggle`): `TextInputLayout` con `app:passwordToggleEnabled="true"` permite alternar la visibilidad de la contraseña.

Entrada multilineal (`multiline`): `TextInputEditText` con `inputType="textMultiLine"` para admitir varias líneas de texto.

Selector (`dropdown`): `Widget.MaterialComponents.TextInputLayout.FilledBox.ExposedDropDownMenu` para crear un menú desplegable expuesto.

Texto decorado (`filled`): `Widget.MaterialComponents.TextInputLayout.FilledBox` proporciona un aspecto de caja rellena.

Contraer el hint (`collapsedHint`): `TextInputLayout` con `app:hintEnabled="false"` para ocultar el hint cuando se escribe.

Contador de caracteres (`counter`): `TextInputLayout` con `app:counterEnabled="true"` muestra un contador de caracteres.

Estilos de Material para TextInputLayout

En Android, los estilos de Material Design proporcionan una forma consistente y visualmente atractiva de diseñar componentes de la interfaz de usuario.

Veremos ejemplos de estilos para TextInputLayout.

Estilo para Bordes Externos (OutlinedBox)

```
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/textInputLayout"
    style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Ingrese su texto">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/textInputEditText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</com.google.android.material.textfield.TextInputLayout>
```

Estilo para Bordes Resaltados (FilledBox)

```
<com.google.android.material.textfield.TextInputLayout  
    style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:hint="Ingrese su texto">  
  
    <com.google.android.material.textfield.TextInputEditText  
        android:id="@+id/textInputEditText"  
        android:layout_width="match_parent"  
        android:layout_height="wrap_content" />  
</com.google.android.material.textfield.TextInputLayout>
```

Estilo de Caja Compacta (Filled Box Dense)

```
<com.google.android.material.textfield.TextInputLayout
    style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox.Dense"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Ingresa su texto">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/textInputEditText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</com.google.android.material.textfield.TextInputLayout>
```

Estos son ejemplos básicos, y puedes personalizar aún más los estilos según tus necesidades. Asegúrate de incluir las bibliotecas de Material Components en tu archivo build.gradle para acceder a estos estilos:

```
dependencies {  
  
    implementation 'androidx.core:core-ktx:1.10.1'  
    implementation 'androidx.appcompat:appcompat:1.6.1'  
    implementation 'com.google.android.material:material:1.9.0'  
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'  
    implementation 'androidx.navigation:navigation-fragment-ktx:2.7.0'  
    implementation 'androidx.navigation:navigation-ui-ktx:2.7.0'
```

Validación

Si la validación de un campo es especialmente compleja podemos crear una función que sea la encargada de validar:

```
fun validName(e:EditText):Boolean{  
    var isValid = true  
    if(e.text.length<2){  
        e.error = "El nombre ha de tener como mínimo dos caracteres"  
        isValid = false  
    }  
    return isValid  
}
```

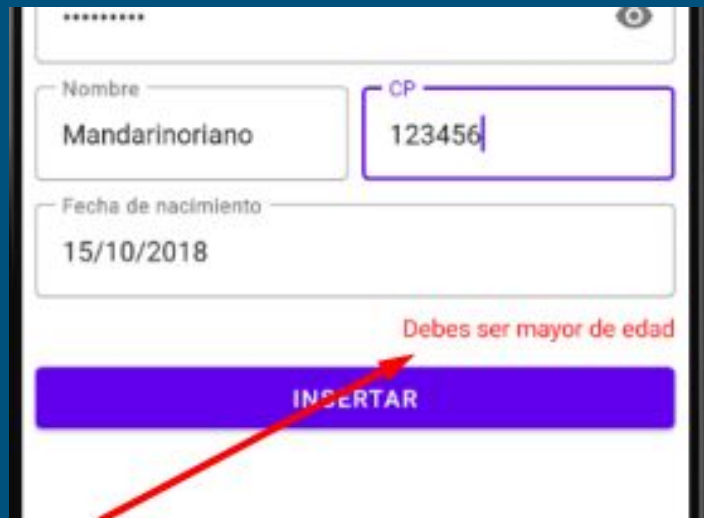
Supongamos que tienes un campo que debe cumplir con ciertas reglas específicas, como tener al menos 8 caracteres, incluir al menos una letra mayúscula, una letra minúscula y un número. Creamos una función de validación específica para este campo:

```
private fun validateComplexField(input: String): Boolean {
    val regex = Regex("(?=.*[a-z])(?=.*[A-Z])(?=.*\\d){8,}\\$")

    return if (regex.matches(input)) {
        // La validación es exitosa
        true
    } else {
        // La validación falla, mostramos un mensaje de error
        showToast("El campo debe tener al menos 8 caracteres, una letra may
        false
    }
}
```

Mensaje de error personalizados

Podemos personalizar el estilo de nuestros mensaje de error estableciendo objetos TextView como visibles o no



A screenshot of a mobile application form. The form contains three input fields: 'Nombre' (Name) with the value 'Mandarinoriano', 'CP' (Postal Code) with the value '123456', and 'Fecha de nacimiento' (Date of Birth) with the value '15/10/2018'. Below these fields is a red error message: 'Debes ser mayor de edad' (You must be of legal age). A red arrow points from the bottom left towards the error message. At the bottom of the form is a blue button labeled 'INSERTAR'.

Personalizar el mensaje nativo

Para personalizar el mensaje de error en un `TextInputLayout` en Android, puedes utilizar el atributo `app:errorEnabled` y el método `setError()` de `TextInputLayout`.

```
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/textInputLayout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:errorEnabled="true">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/editText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Ingrese su nombre" />

</com.google.android.material.textfield.TextInputLayout>
```

En tu código Kotlin, puedes establecer o quitar el mensaje de error según sea necesario.

```
val textInputLayout = findViewById<TextInputLayout>(R.id.textInputLayout)
val editText = findViewById<TextInputEditText>(R.id.editText)

// Validar el campo y mostrar el mensaje de error si es necesario
if (nombre.isEmpty()) {
    textInputLayout.error = "Por favor, ingrese su nombre"
} else {
    // Si el campo es válido, quitar el mensaje de error
    textInputLayout.error = null
}

// Opcional: También puedes establecer un mensaje de ayuda (helper text)
textInputLayout.helperText = "Este es un campo opcional"
```

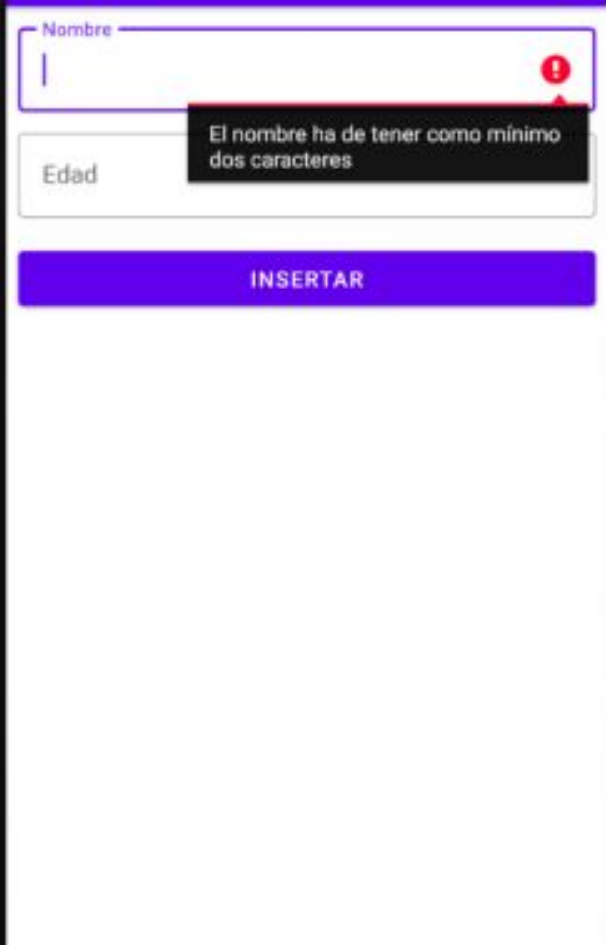
Si establecemos una función de validación para cada input podemos crear un método genérico para validar cualquier formulario

Luego, puedes utilizar esta función en tu método general de validación (**validateForm()**), llamándola con el valor del campo que desees validar de manera especial:

```
private fun validateForm(): Boolean {  
    var isValid = true  
  
    // Validar campo de nombre  
    val name = editTextName.text.toString().trim()  
    if (name.isEmpty()) {  
        textInputLayoutName.error = "Ingrese su nombre"  
        isValid = false  
    } else {  
        textInputLayoutName.error = null  
    }  
  
    // Validar campo de correo electrónico  
    val email = editTextEmail.text.toString().trim()  
    if (email.isEmpty() || !Patterns.EMAIL_ADDRESS.matcher(email).matches())  
        textInputLayoutEmail.error = "Ingrese un correo electrónico válido"  
        isValid = false  
    } else {  
        textInputLayoutEmail.error = null  
    }  
  
    // Validar campo complejo  
    val complexField = editTextComplexField.text.toString().trim()  
    if (!validateComplexField(complexField)) {  
        isValid = false  
    }  
  
    return isValid  
}
```

Ejercicio 1

- Crea una app con un formulario con dos campos: nombre y edad • El nombre es requerido y deberá tener como mínimo dos caracteres
- La edad no es obligatoria, pero si se introduce se debe ser mayor de edad
- Si el formulario es correcto volver a MainActivity



The screenshot shows a mobile application interface with a white background. At the top, there is a form with two input fields. The first field is labeled 'Nombre' and contains a single character 'I'. A red error message box is overlaid on the right side of this field, stating 'El nombre ha de tener como mínimo dos caracteres'. The second field is labeled 'Edad' and is empty. Below the form is a large red button with the text 'INSERTAR' in white capital letters.

Ejercicio 2

Crea una aplicación Android que solicite al usuario información básica utilizando `TextInputLayout` y `TextInputEditText`. La aplicación debe validar los campos antes de permitir el envío del formulario.

Requisitos:

- Un campo para el nombre del usuario.
- Un campo para el correo electrónico.
- Un campo para la contraseña.
- Un botón de envío.

Funcionalidades Adicionales (opcional):

- Mostrar un mensaje de error personalizado para cada validación fallida.
- Utilizar diferentes estilos de `TextInputLayout` para resaltar los campos según su estado (correcto/incorrecto).

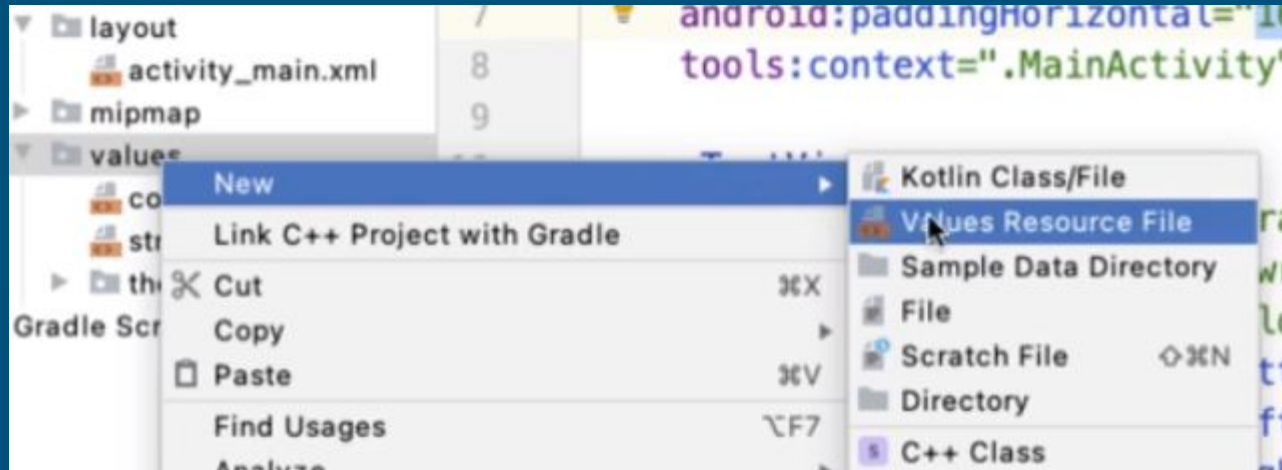
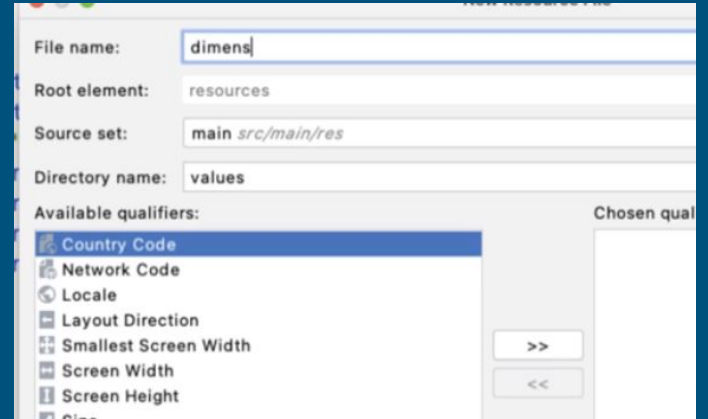
Validaciones:

- El nombre no puede estar vacío.
- El correo electrónico debe ser válido (contener "@" y al menos un punto).
- La contraseña debe tener al menos 6 caracteres.

Recursos “dimen”

En Android, los recursos **dimens** en XML se utilizan para definir valores de dimensiones como tamaños de texto, márgenes, rellenos, alturas, anchuras, etc.

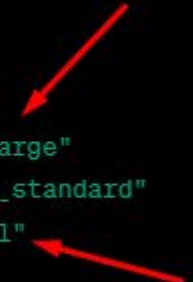
En el archivo res/values/dimens.xml,
defines tus recursos dimsens



Puedes utilizar estos recursos en tus archivos de diseño XML

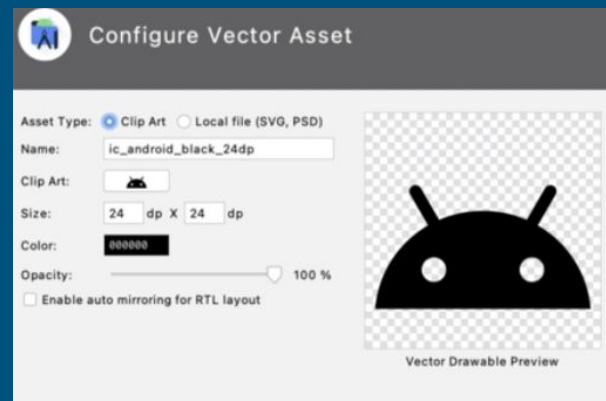
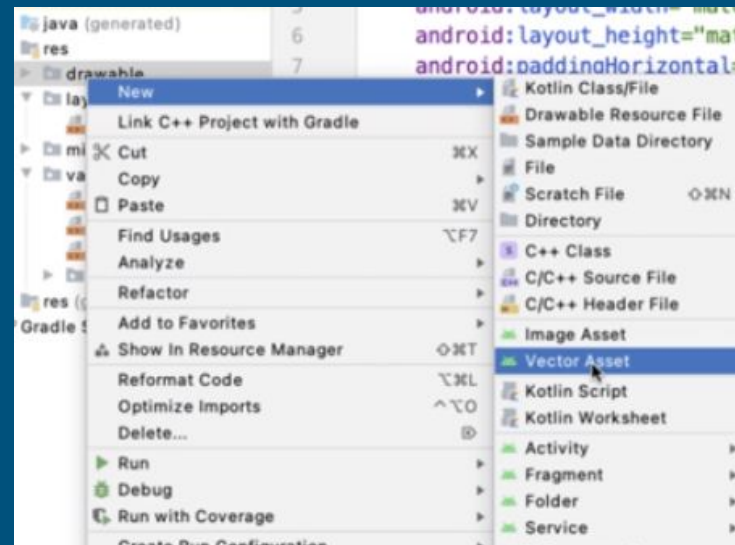
```
<!-- res/values/dimens.xml -->
<resources>
    <dimen name="text_size_large">20sp</dimen>
    <dimen name="margin_standard">16dp</dimen>
    <dimen name="padding_small">8dp</dimen>
</resources>
```

```
<!-- res/layout/activity_main.xml -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textSize="@dimen/text_size_large"
    android:layout_margin="@dimen/margin_standard"
    android:padding="@dimen/padding_small"
    android:text="Hello, World!" />
```



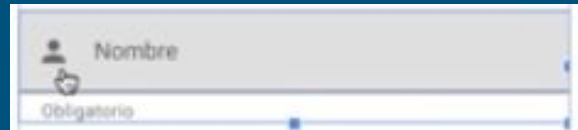
Iconos en los editText

Creamos los iconos desde Android studio



Usamos el atributo de icono

```
android:layout_height="wrap_content"  
android:layout_marginTop="@dimen/common_padding_def  
android:hint="Nombre"  
app:helperText="Obligatorio"  
app:startIconDrawable="@drawable/ic_person"  
style="@style/Widget.MaterialComponents.TextInputLa  
com.google.android.material.textfield.TextInputEdi
```



Varios inputType

Aquí por ejemplo añadimos el filtro **textCapWord** para que el teclado ponga la primera mayúscula siempre

```
<com.google.android.material.textfield.TextInputEditText  
    android:id="@+id/etName"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:inputType="textPersonName|textCapWords"/>
```



Límite de caracteres

El atributo será maxLength

```
style="@style/Widget.MaterialComponents.TextInputLayout.Outline
<com.google.android.material.textfield.TextInputEditText
    android:id="@+id/etName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="textPersonName|textCapWords"
    android:maxLength="@integer/name_max_length"/>
</com.google.android.material.textfield.TextInputLayout>
```

Hemos creado el archivo integers.xml

Resource value: 30

Source set: main/src/main/res

File name: integers.xml

Create the resource in directories:

- ☒ values
- ☐ values-night

```
android:inputType="textPersonName|textCapWords"
android:maxLength="@integer/name_max_length"/>
</com.google.android.material.textfield.TextInputLayout>
<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="@dimen/common_padding_default"
    android:hint="Apellidos">
```

- Create integer value resource 'name_max_length' ▶
- ✓ Override Resource in Other Configuration...
- ✓ Rearrange tag attributes
- ✓ Remove attribute
- ✓ Inject language or reference

Press ⌘Space to open preview

Propiedad **focusable**

Esto deshabilita el click normal dentro del componente.

También puedes establecer el foco programáticamente en Kotlin utilizando el método **isFocusable**.

```
<com.google.android.material.textfield.TextInputLayout
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="60"
    android:layout_marginTop="@dimen/common_padding_default"
    android:layout_marginStart="@dimen/common_padding_min"
    android:hint="Fecha de nacimiento"
    app:startIconDrawable="@drawable/ic_calendar_today"
    style="@style/Widget.MaterialComponents.TextInputLayout.Outl
    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/etDateBirth"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:focusable="false"/>
</com.google.android.material.textfield.TextInputLayout>
```

```
val myButton: Button = findViewById(R.id.myButton)
myButton.isFocusable = true
```

Autocomplete



Es una mezcla entre un autocomplete y un spinner (un select de html de toah la vida)

Usaremos el elemento “**AppCompatAutoCompleteTextView**” ya que a pesar de haber otros, el equipo de Android recomienda ese por las compatibilidades.

Al principio no tendrá función porque necesita un array de datos para mostrar, sino será un editText clásico

```
<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="País"
    android:layout_marginTop="@dimen/common_padding_default"
    style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox.ExposedDropdownMenu"
    <androidx.appcompat.widget.AppCompatAutoCompleteTextView
        android:id="@+id/actvCountries"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
</com.google.android.material.textfield.TextInputLayout>
```

AutoComplete Kotlin configuración

Tenemos que crearle un adaptador al que le pasaremos el **contexto**, un **recurso** que maneje la vista y el **array** de datos.

```
val countries = arrayOf("Argentina", "Bolivia", "Chile", "Colombia", "Ecuador", "España",  
    "Estados Unidos", "México", "Panamá", "Peru", "Uruguay")  
val adapter = ArrayAdapter(context: this, android.R.layout.simple_spinner_dropdown_item,  
    countries)  
  
binding.actvCountries.setAdapter(adapter)]
```

android.R.layout accede a los recursos que ya contiene el SO de android

Manejar Selecciones del autoComplete

Habr  que poner un listener en los elementos del autoComplete para que detecte la selecci n, usaremos el segundo m todo:

```
binding.actvCountries.setOnI
```

```
binding.etDateBirth.  
val builder = M
```

```
onItemClickListener(l: ((AdapterView... Unit  
onItemClickListener { AdapterView<*>?, View?  
onItemClickListener AdapterView.OnItemClickListener
```

```
binding.actvCountries.setOnItemClickListener { adapterView, view, i, l ->  
    Toast.makeText(context: this, countries.get(i), Toast.LENGTH_SHORT).show()  
}
```

Aqu  lanzamos un toast accediendo al array de datos con el  ndice "i"

Otra forma

Los guiones bajos _ son marcadores de posición para argumentos que no se utilizan en el cuerpo de la función lambda. En este caso, estamos interesados en el tercer argumento, position, que representa la posición del elemento seleccionado en la lista.

```
autoCompleteTextView.setOnItemClickListener { _, _, position, _ ->
    val selectedText = adapter.getItem(position).toString()
    // Hacer algo con el texto seleccionado
}
```

val selectedText = adapter.getItem(position).toString()

Obtiene el texto del elemento seleccionado en la posición dada

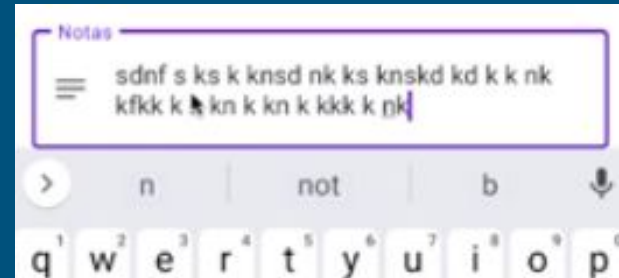
Hacer focus en otro elemento desde un input

Si no lo hacemos, al pulsar en “intro” de teclado nos pondrá un salto de linea.

```
binding.actvCountries.setOnItemClickListener {  
    binding.etPlaceBirth.requestFocus()  
    Toast.makeText(context, this, countries  
}
```

Tipo de input “textMultiLine”

```
<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="@dimen/common_padding_default"
    android:hint="Notas"
    app:startIconDrawable="@drawable/ic_notes"
    style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedE
<com.google.android.material.textfield.TextInputEditText
    android:id="@+id/etNotes"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="textMultiLine"
    android:maxLength="@integer/notes_max_length"/>
</com.google.android.material.textfield.TextInputLayout>
```



Text field con counter

Necesitamos los atributos **counterEnabled** para habilitarlo y **counterMaxLength** para restringirlo.

```
<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="@dimen/common_padding_default"
    android:hint="@string/hint_notes"
    app:counterEnabled="true"
    app:counterMaxLength="@integer/notes_max_length"
    app:startIconDrawable="@drawable/ic_notes"
    style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedE
<com.google.android.material.textfield.TextInputEditText
    android:id="@+id/etNotes"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="textMultiLine"
```

Notas

nks nksksk a knk knk kas kasnk sn knk
nask nsknsk nkas nkn ksan ks nksn kns
kn kn kns kasn kn knksan knkkn kansk
ksndkh

120/120

Como hacer que de un input vayamos a otro

Usaremos los atributos
imeOptions y **nextFocusForward**
uno habilita la acción y otro dice a
qué elemento queremos ir al
completar el campo actual

```
style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
<com.google.android.material.textfield.TextInputEditText
    android:id="@+id/etHeight"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:inputType="number"
    android:maxLength="@integer/height_max_length"
    android:imeOptions="actionNext"
    android:nextFocusForward="@id/etPlaceBirth"/>
</com.google.android.material.textfield.TextInputLayout>
```

A diagram illustrating the focus flow between two text input fields. Two red arrows originate from the first field (etHeight) and point towards the second field (etPlaceBirth). One arrow points to the 'imeOptions="actionNext"' attribute, and the other points to the 'nextFocusForward="@id/etPlaceBirth"' attribute, indicating how these attributes facilitate moving focus to the next field.

DatePicker

DatePicker

Aunque existe el tipo 'date' para los EditText este no abre un DatePicker

Para que al pulsar un campo aparezca un DatePicker hace falta gestionarlo de forma manual

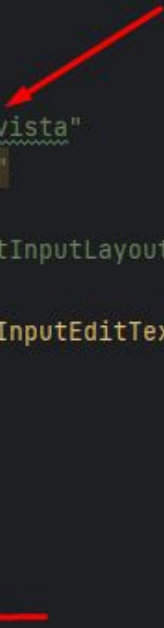
Con la introducción de `MaterialDatePicker` en la biblioteca de Material Components, gestionar fechas se ha vuelto más sencillo.

1 paso

En tu archivo de diseño
XML:

```
<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_marginBottom="10dp"
    android:id="@+id/layout_edit_text_fecha_prevista"
    android:hint="Fecha prevista de realización"
    app:helperText="Obligatorio"
    style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
>

    <com.google.android.material.textfield.TextInputEditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="date"
        android:fontFamily="@font/futura"
        android:textSize="20dp"
        android:id="@+id/fecha_prevista"
        android:focusable="false"
    />
</com.google.android.material.textfield.TextInputLayout>
```



2 paso

En tu archivo de código
Kotlin:

```
binding.fechaPrevista.setOnClickListener { It: View!  
    val builder = MaterialDatePicker.Builder.datePicker()  
    val picker = builder.build()  
  
    picker.addOnPositiveButtonClickListener { It: Long!  
        binding.fechaPrevista.setText(picker.headerText)  
    }  
  
    picker.show(supportFragmentManager, tag: "jeje")  
}
```

Extras

```
// Configurar el MaterialDatePicker
materialDatePicker = MaterialDatePicker.Builder.datePicker()
    .setTitleText("Seleccionar fecha")
    .build()
```



```
// Asignar el MaterialDatePicker al EditText
editTextDate.setOnClickListener {
    materialDatePicker.show(supportFragmentManager, materialDatePicker.toString())
}
```

Ejercicio 3

Crea una app con el siguiente formulario:

Validación:

- Correo requerido y válido
- Contraseña requerida, más de siete caracteres y debe contener al menos un número, una letra mayúscula y una minúscula
- CP no requerido pero de insertarse debe ser válido
- El usuario debe ser mayor de edad

Formulario de registro:

- Correo:
- Contraseña:
- Nombre:
- CP:
- Fecha de nacimiento:
- Botón: INSERTAR

AlertDialog

Un AlertDialog en Android es un componente de interfaz de usuario que se utiliza para mostrar un mensaje o realizar una acción específica.

1. Se crea un AlertDialog.Builder y se configuran el título y el mensaje del diálogo.
2. Se configuran botones (positivo y negativo) junto con sus acciones asociadas.
3. Se muestra el AlertDialog mediante el método show().

```
// Crear un AlertDialog.Builder
val builder = AlertDialog.Builder(context, this)

// Configura el título y el mensaje del AlertDialog
builder.setTitle("Título del Diálogo")
    .setMessage("Este es un mensaje en el cuerpo del diálogo.")

// Configura botones y su comportamiento
builder.setPositiveButton(text: "Aceptar") { dialog, _ ->
    // Acción al hacer clic en el botón "Aceptar"
    dialog.dismiss() // Cierra el diálogo
}

builder.setNegativeButton(text: "Cancelar") { dialog, _ ->
    // Acción al hacer clic en el botón "Cancelar"
    dialog.dismiss() // Cierra el diálogo
}

// Mostrar el AlertDialog
val alertDialog: AlertDialog = builder.create()
alertDialog.show()
```

Limpiando formularios

clear()

Para limpiar formularios en Android, puedes utilizar el método `clear()` en las vistas de los elementos del formulario, como `EditText`.

```
editTextNombre.text.clear()  
editTextCorreo.text.clear()
```

Formatear Fechas

En Android, puedes utilizar SimpleDateFormat para formatear fechas en combinación con el componente DatePicker.

```
// Formatear la fecha con SimpleDateFormat
val formatoFecha = SimpleDateFormat(pattern: "dd/MM/yyyy", Locale.getDefault())
val fechaFormateada = formatoFecha.format(time)
```

Evitar desajustes horarios y ejemplo completo

La línea de `timeZone = ...` se usa porque a veces con los emuladores hay un desajuste de tiempo en función de la zona horaria y puede darnos una fecha errónea como un día menos o cosas similares

```
binding.etDateBirth.setOnClickListener { it: View!
    val builder = MaterialDatePicker.Builder.datePicker()
    val picker = builder.build()

    picker.addOnPositiveButtonClickListener { timeInMilliseconds ->
        val dateStr = SimpleDateFormat( pattern: "dd/MM/yyyy",
            Locale.getDefault()).apply { this: SimpleDateFormat
                timeZone = TimeZone.getTimeZone( id: "UTC")
            }.format(timeInMilliseconds)

        binding.etDateBirth.setText(dateStr)
    }

    picker.show(supportFragmentManager, picker.toString())
}
```

